

Atividade Prática: Coordenação entre Processos

Sistemas Operacionais

Prof. Pedro Botelho

29 de abril de 2026

1. (6 pontos) Um problema clássico na sincronização de processos é o problema **Produtor-Consumidor**. Esse problema envolve dois ou mais processos (ou threads): um ou mais **produtores** que geram dados e os colocam em um *buffer* (uma região de memória, como um vetor ou uma lista), e um ou mais **consumidores** que retiram dados desse *buffer*. O desafio principal é garantir que o produtor não tente adicionar dados a um *buffer* cheio e que o consumidor não tente retirar dados de um *buffer* vazio, evitando condições de corrida e inconsistências de dados. Abaixo estão os requisitos do projeto:

- Seu trabalho será projetar um sistema com múltiplas *threads* de um **gerenciador centralizado de caixas** de uma loja. Sempre que um caixa se conecta ao servidor, uma *thread* é criada para gerenciá-lo. De forma a simplificar, o sistema já inicia com os três caixas conectados (devendo apenas criar suas *threads*), além da *thread* de gerenciamento, não sendo preciso tratar conexão, apenas a sincronização entre *threads*. Sendo assim, existem 9 *threads*: 6 produtoras e 2 consumidoras. A única função da *thread* principal (ou despachante) é criar as *threads* produtoras e consumidoras e esperar que elas terminem.
- Um *buffer* de 5 espaços deve ser usado (um vetor simples pode ser usado com índice circular), de forma que as *threads* produtoras possam inserir os valores das vendas e a consumidora possa processá-lo (não esqueça de gerenciar seu espaço usando um **semáforo**). O processamento deverá ser sincronizado, de forma que o acesso ao *buffer* seja protegido (via **mutex**). Ainda, a *thread* consumidora deve ser “avisada” pelas *threads* produtoras sempre que dados novos forem inseridos no *buffer*, para que possa processá-los (Dica: Usem semáforos).
- Cada uma das ***threads* produtoras** deve produzir 20 valores relativos ao preço dos produtos, entre 1 a 1000 reais. Para isso, use valores aleatórios, via funções **srand()** e **rand()**, colocando-os no *buffer* (se não cheio), devendo sinalizar à *thread* consumidora e imprimir na tela o valor do produto, seu TID (*thread* ID) e o número da iteração atual, indicando também que é produtora. Ao final, as *threads* produtoras devem esperar um certo tempo antes de produzirem novos valores (use valores aleatórios como *delay*, entre 1 e 5 segundos). Quando as *threads* produtoras produzirem todos os valores, devem finalizar e informar à *thread* consumidora que finalizaram (Dica: Use ***pthread_cond***).

- A *thread* **consumidora** irá consumir os valores postos no *buffer* compartilhado quando as *threads* produtoras sinalizarem, porém com restrições: essa *thread* irá retirar os 5 elementos do *buffer* (quando o buffer estiver cheio) e realizar uma média aritmética. Assim, irá imprimir a média na tela, junto a seu TID e a iteração corrente, voltando a esperar o *buffer* ficar cheio novamente.
- Abaixo está um exemplo da estrutura geral do código:

```

1 void *producer(void *args) {
2
3     for(int i = 0; i < 20; i++) {
4         // acessar buffer compartilhado (produzir)
5         // imprimir TID/dados
6         // esperar por um tempo aleatorio
7     }
8     // imprimir que finalizou
9     // sinalizar que finalizou
10 }
11
12 void *consumer(void *args) {
13     while( /* tem produtoras ainda? */ ) {
14         // esperar 5 dados
15         // acessar buffer compartilhado (consumir)
16         // imprimir TID/dados
17     }
18     // imprimir que finalizou
19 }
20
21 int main(void) {
22     // criar threads produtoras
23     // criar thread consumidora
24     // esperar threads terminarem
25 }

```

- Abaixo está um exemplo de saída do programa. É importante que as *threads* imprimam uma mensagem informando que finalizaram:

```

1 // ...
2 // (P) TID: 1000 | VALOR: R$ 210 | ITERACAO: 25
3 // (P) TID: 1001 | VALOR: R$ 32 | ITERACAO: 20
4 // (P) TID: 1001 finalizou
5 // (P) TID: 1002 | VALOR: R$ 5 | ITERACAO: 12
6 // (C) TID: 1003 | MEDIA: R$ 620 | ITERACAO: 30
7 // ...

```

Boa sorte!